Tunify

Introduction:

Tunify is a modern music-streaming application built using React.js, designed to give users a smooth and immersive listening experience. The app blends a clean interface with strong functionality, making it easy to discover tracks, manage playlists, and enjoy music without interruptions.

Because Tunify's interface is powered by React, the application stays fast, responsive, and visually appealing across various devices. Whether you're browsing new genres or organizing songs you love, the platform adapts to your preferences and provides a consistent experience.

At its core, Tunify utilizes React to deliver an interactive layout that feels natural to navigate. Regardless of whether you're on mobile or desktop, the app focuses on delivering uninterrupted playback and effortless music exploration.

Scenario-Based Intro:

Picture a regular evening when you feel like relaxing with music. You open Tunify, and the moment it loads, a soft and vibrant interface welcomes you. You head to the **Browse** section and instantly see a neatly curated collection of tracks. Searching for *Chill Songs* brings up a list of calm, soothing music.

You tap on the first track, and the bottom music player smoothly pops up. It lets you change volume, skip to the next song, replay, or loop your favorites. You like the current track and save it to your **Liked List**. Every action feels quick and seamless, making the entire experience pleasant.

Target Audience:

Tunify is mainly designed for:

Music Lovers

People who enjoy exploring fresh tracks and maintaining their personal music library.

Playlist Creators

Users who love arranging music into custom playlists for different moods or occasions.

Project Goals and Objectives:

The main purpose of Tunify is to deliver a high-quality, enjoyable music-streaming experience. The project intends to:

- **Offer an easy-to-use interface** where users can navigate, browse, and play music effortlessly.
- **Provide a feature-rich audio player** that supports shuffling, looping, queue management, and smooth track transitions.
- **Utilize a modern development stack**, including React.js, Vite, and Tailwind CSS, ensuring both appearance and performance stay top-notch.

Key Features:

- **Live Audio Player**
  A persistent player that keeps running even when the user switches between pages. It includes essential controls like Play/Pause, Next, Previous, Loop, and Shuffle.
- **Music Discovery**
  Users can browse songs through different categories and genres.
- **User Authentication**
  Secure login and registration allow users to maintain their preferences and liked songs.
- **Personalized Library**
  Users can create playlists, save favorite songs, and organize music the way they like.

PRE-REQUISITES:

Here are the key prerequisites for developing or running this application:

- **Node.js and npm**

Node.js is required to run the development environment.

Download: nodejs.org

- **React.js & Vite**

This project is built using React with Vite for a fast development experience.

Create a new app: npm create vite@latest

Navigate to the project directory:

cd tunify/client

npm install

Running the React App:

With the React app created, you can now start the development server and see your React application in action.

Start the development server:

npm run dev

This command launches the development server, and you can access your React app at http://localhost:5173 in your web browser.

- **Run the Mock Backend Server:**

Create the JSON extension file to store the site data. For example, “db.json”

To run this mock backend server:

npx json-server db.json

- **Dependencies:**

Key libraries used in this project:

- **react-router-dom:** For client-side routing.
- **tailwindcss:** For utility-first styling.
- **json-server:** For a mock backend API.
- **axios:** For making HTTP requests.
- **Frame-motion**: To make animations in the UI.

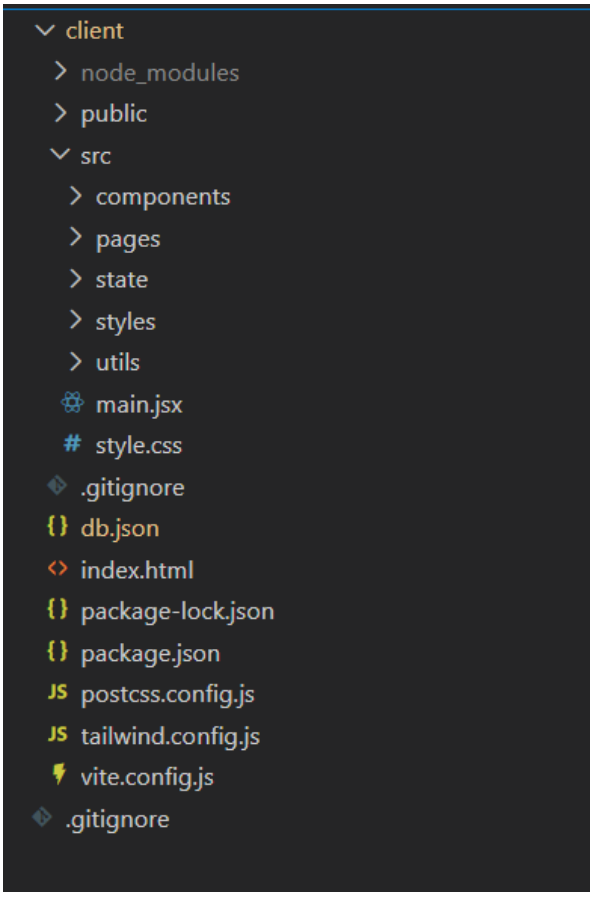
➢ **Version Control:** Used Git for version control, enabling collaboration and tracking changes throughout the development process. Platforms like GitHub or Bitbucket can host your repository.

• Git: Download and installation instructions can be found at: https://git-scm.com/downloads

➢ **Development Environment**: Choose a code editor or Integrated Development Environment (IDE) that suits your preferences, such as Visual Studio Code, Sublime Text, or WebStorm.

• Visual Studio Code: Download from https://code.visualstudio.com/download

Project Structure:



The project is organized to promote maintainability and scalability:

- client/src/main.jsx: The entry point of the application. It handles the root rendering and sets up the global providers (AuthProvider, PlayerProvider) and the AppRouter.
- **client/src/components/:** Contains reusable UI components like Navbar, PlayerModal, SongCard, etc.
- **client/src/pages/:** Contains page-level components like HomePage, BrowsePage, LoginPage, etc.
- **client/src/state/:** Contains Context API files for global state management (AuthContext, PlayerContext).
- **client/src/utils/:** Utility functions and API configuration.

PROJECT FLOW: -

Project demo:

Before starting to work on this project, let's see the demo.

Demo link:

https://drive.google.com/file/d/1Jkbc54EFcRC5H0jQveXd28IdXHeNZEny/view?usp=drive_link

Use the code in:

https://github.com/ganesh-gonuguntla/tunify-react-frontend-app-course-project.git

Code Explanation:

https://drive.google.com/file/d/1jGKYJ0FA3IBdxGxGAsE3GJdqZuJcBB2c/view?usp=sharing

Code Description:

Our project has lot of components, these are some of them,

main.jsx (Entry Point)

```jsx
import './style.css'
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom'

import LoginPage from './pages/LoginPage'
import HomePage from './pages/HomePage'
import CategoryPage from './pages/CategoryPage'
import ProfilePage from './pages/ProfilePage'
import RegisterPage from './pages/RegisterPage'
import LikedSongsPage from './pages/LikedSongsPage'
import BrowsePage from './pages/BrowsePage'
import Navbar from './components/Navbar'
import PlayerModal from './components/PlayerModal'
import { AuthProvider, useAuth } from './state/AuthContext'
import { PlayerProvider } from './state/PlayerContext'

function AppRouter() {
  const { user } = useAuth()
  const isAuthed = Boolean(user)

  return (
    <BrowserRouter>
      {isAuthed && <Navbar />}
      {isAuthed && <PlayerModal />}
      <Routes>
        <Route path="/login" element={<LoginPage />} />
        <Route path="/register" element={isAuthed ? <Navigate to="/" replace /> : <RegisterPage />} />
        <Route path="/" element={isAuthed ? <HomePage /> : <Navigate to="/login" replace />} />
        <Route path="/category/:slug" element={isAuthed ? <CategoryPage /> : <Navigate to="/login" replace />} />
        <Route path="/browse" element={isAuthed ? <BrowsePage /> : <Navigate to="/login" replace />} />
        <Route path="/liked" element={isAuthed ? <LikedSongsPage /> : <Navigate to="/login" replace />} />
        <Route path="/profile" element={isAuthed ? <ProfilePage /> : <Navigate to="/login" replace />} />
        <Route path="*" element={<Navigate to={isAuthed ? '/' : '/login'} replace />} />
      </Routes>
    </BrowserRouter>
```

Code Description:

This file serves as the root of the React application.

- **Router**: It uses BrowserRouter to manage navigation.
- **Routes**: Defines paths for /login, /, /browse, /liked, etc., and handles protected routes (redirecting unauthenticated users to login).

- **Providers**: Wraps the application in AuthProvider and PlayerProvider to ensure authentication state and player state are accessible globally.

PlayerContext.jsx (State Management)

This context manages the entire audio playback logic.

Code Description:

- **State**: Tracks queue, currentIndex, isPlaying, volume, shuffle, and loop.
- **Audio Ref**: Uses a useRef to hold the Audio object, allowing direct control over playback without re-rendering unnecessarily.
- **Functions**:
    - playSongs(songs, startIndex): Replaces the queue and starts playing a new list of songs.
    - next () / prev(): Handles navigation through the queue, respecting shuffle mode.
    - play () / pause (): Controls the current playback state.
- **Effects**: useEffect hooks listen for audio events like ended (to auto-play the next song) and timeupdate (to update the progress bar).

  AppRouter Component (in main.jsx)
- **Conditional Rendering**: Checks if a user is authenticated (isAuthed) to decide whether to show the Navbar and PlayerModal.
- **Route Guards**: Protects private routes like /profile and /liked by checking the user's auth status.

Project Execution:

To run the Tunify application locally:

1. **Install Dependencies**: Navigate to the client directory and install the necessary packages.

   cd client

   npm install
2. **Start the Backend Server**: Run the JSON server to mock the API.

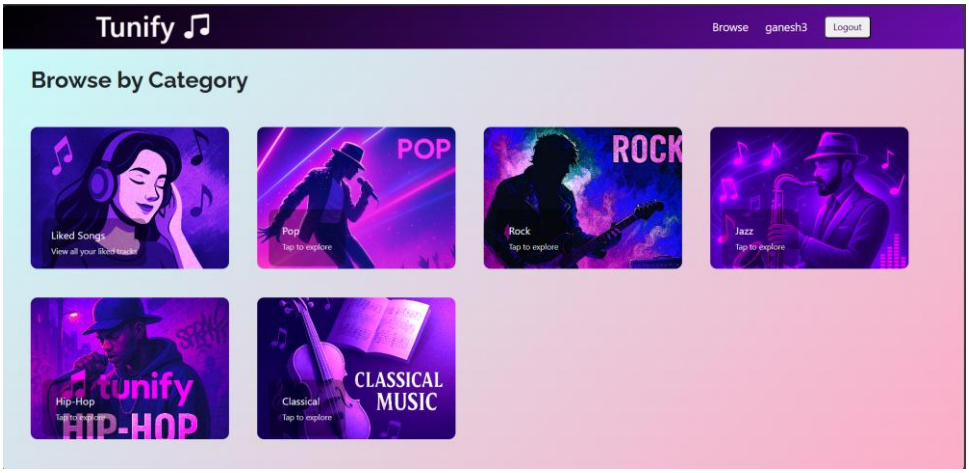
   npx json-server db.json

3. **Start the React App**: Launch the json-server(Mock DataBase).

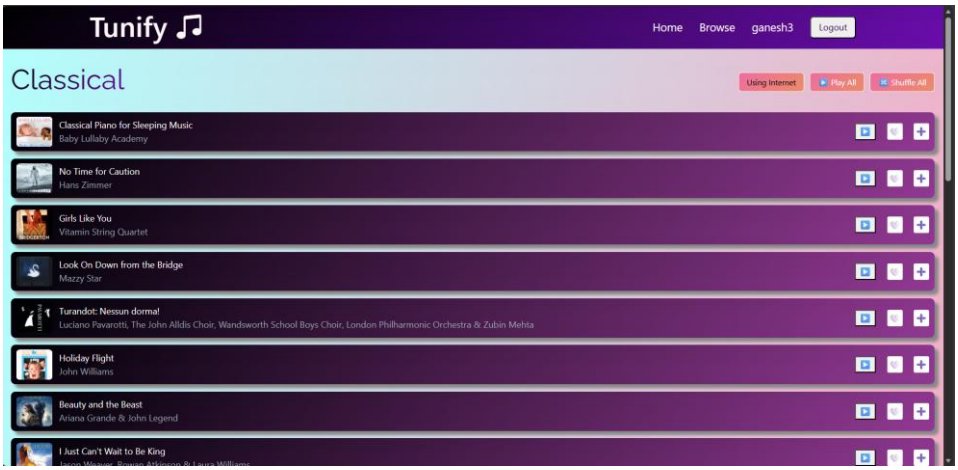
   npm run dev

4. **Access the App**: Open your browser and go to http://localhost:5173 (or the port shown in your terminal).

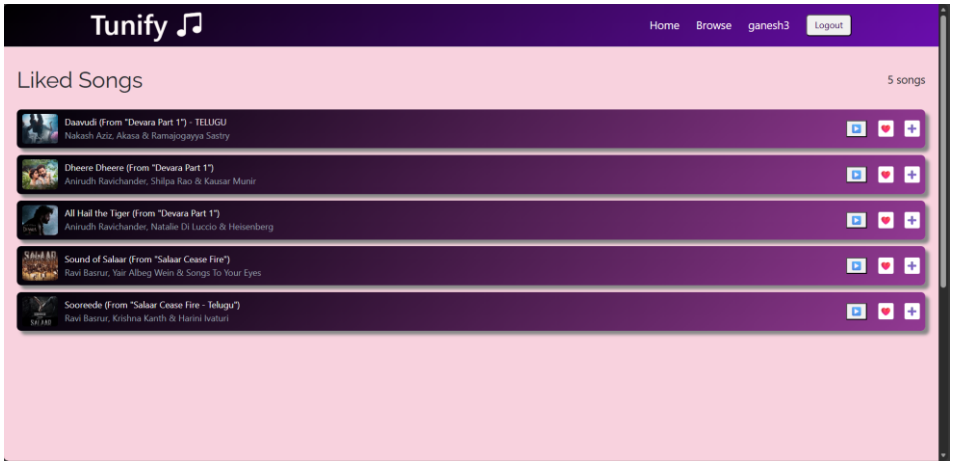
- **Home Page**:



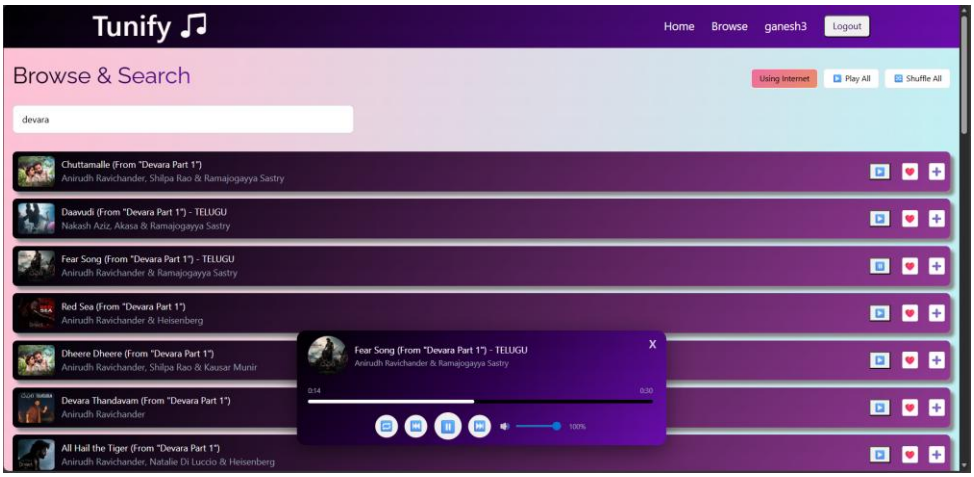
- **Category page:**



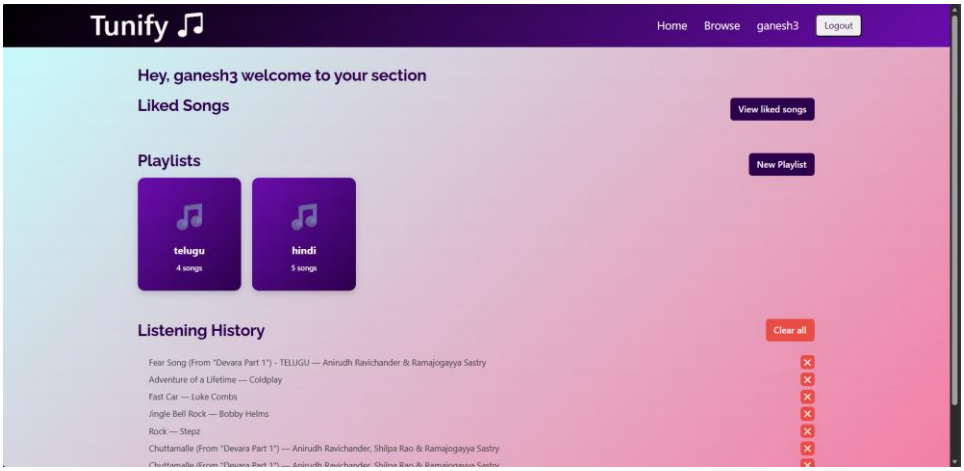
- **Liked Songs Page:**



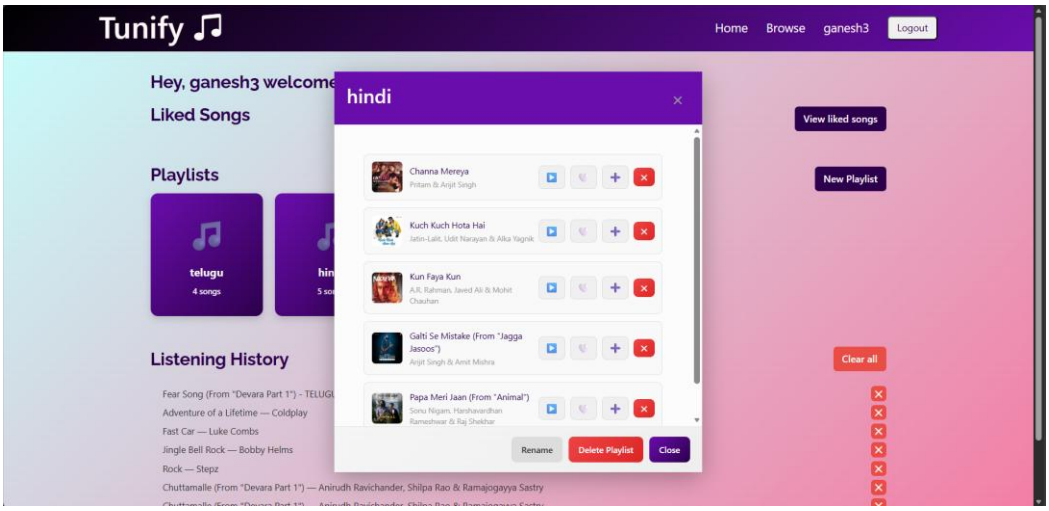
- **Song Player Modal Component and Browser Page:**



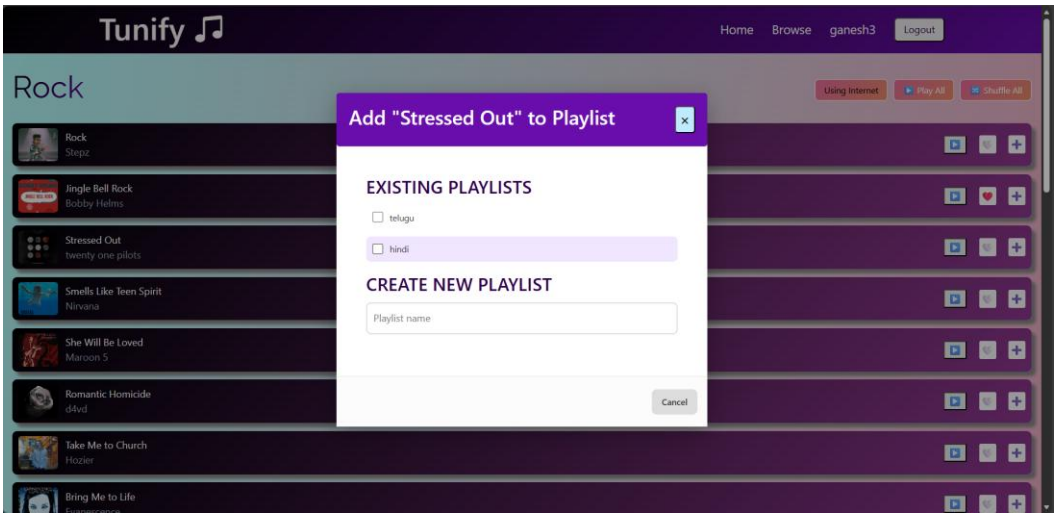
- **Profile Page (includes Playlists,Listening History and redirect button to Liked Page):**



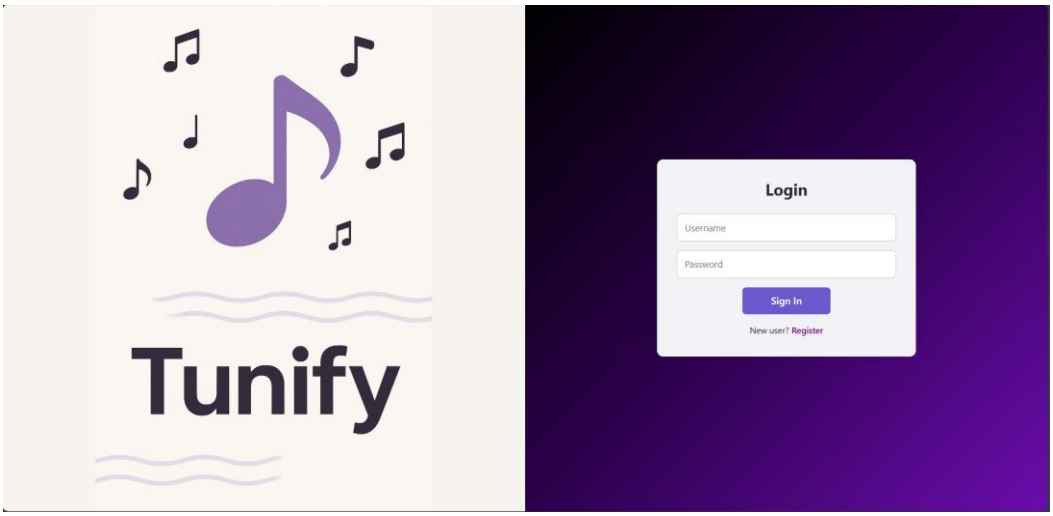
- **Playlist modal:**



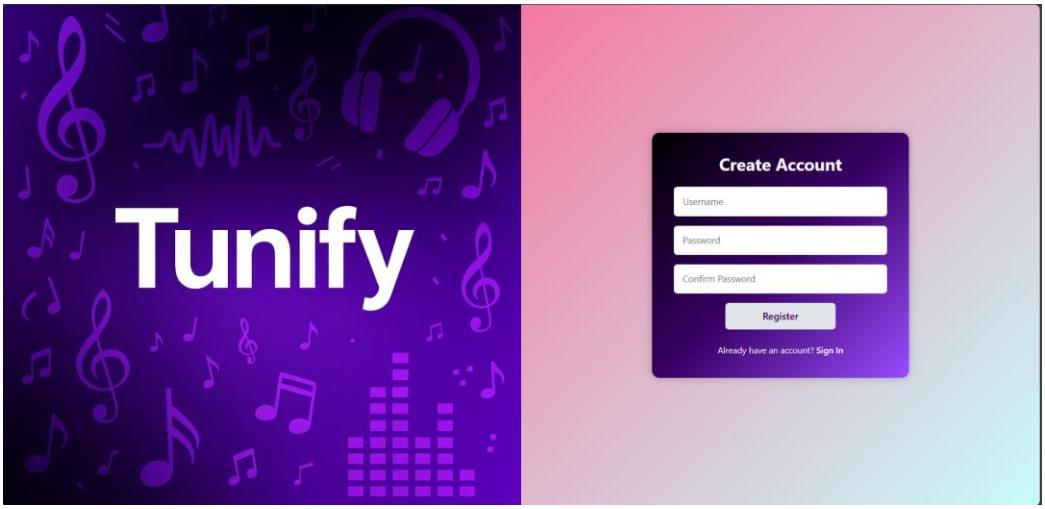
- **Add to Playlist Modal/ Create Playlist Modal:**



- **Login page:**



- **Registration Page:**



Project Demo Link:

https://drive.google.com/file/d/1Jkbc54EFcRC5H0jQveXd28IdXHeNZEny/view?usp=drive_link